

CS:APP Data Lab 位级运算实验报告

姓名：王彦霖

学号：25800190031

实验项目：ICS2026 — Data Lab

实验语言：C

实验文件：bits.c

报告整理日期：2026 年 10 月 9 日

报告说明：本文依据本人提交的 bits.c、逐题思路记录和终端测试截图整理。P11~P17 的分析在原思路记录中未完整给出，现依据实际代码补充。测试结果保留当前版本的真实情况；本次版本尚有两题功能测试未通过、两题操作符数量超限。

一、实验目的

Data Lab 要求在受限的 C 语言表达式中实现整数位运算、算术运算和浮点数位级处理函数。本实验主要练习以下能力：

1. 理解 32 位补码整数的表示方式、符号扩展以及算术右移与逻辑右移的区别。
2. 使用有限的位运算构造掩码、实现选择、比较、取反、循环移位与计数。
3. 理解溢出、舍入和边界值对程序正确性的影响。
4. 理解 IEEE 754 单精度浮点数的符号位、阶码、尾数字段，以及向最近偶数舍入规则。
5. 使用自动化工具分别检查**功能正确性**与**操作符合法性**，明确两种检查并不等价。

二、实验环境与约束

实验在 Ubuntu/Linux 终端中开展，项目目录为 /home/provencep/ICS2026/DataLab。源文件为 bits.c，使用实验提供的 btest 测试功能，用 check_ops.py 统计运算符使用情况。

整数题目采用 32 位补码模型：最高位 bit 31 具有负权重，其余 31 位与符号位共同决定整数的值。整数题通常仅允许 ! ~ & ^ | + << >> 及 0~255 的常量，同时不允许条件、循环、函数调用或额外的数据类型；个别题目进一步限制操作符。浮点题可以使用 if、while、整数比较以及 unsigned，但不能直接使用浮点类型或浮点运算。

在该实验指定的 32 位位向量模型下，对结果只保留低 32 位。对移位量的处理需要格外谨慎，因为位移量若小于 0 或超过 31，结果不符合题目允许的操作范围。

三、各题实现思路与分析

P1 — signMask

目标：返回仅最高位为 1 的掩码，即 `0x80000000`。

32 位整数的最高位编号为 31。从最低位为 1 的整数出发，将其左移 31 位即可将唯一的 1 移到符号位，其他位均为 0。

```
return 1 << 31;
```

这种实现仅用一次移位操作，满足最多 2 次操作的限制。

P2 — bitXor

目标：仅使用 `~` 和 `&` 实现 $x \oplus y$ 。

异或的真值表表明：两位不同时结果为 1，相同时结果为 0。因此可先写为 $(x \& \sim y) \mid (\sim x \& y)$ 。由于题目不允许 `|`，利用德摩根定律 $A \mid B = \sim(\sim A \& \sim B)$ 将按位或消去，得到：

```
return ~(~(x & ~y) & ~(~x & y));
```

表达式共使用 8 个允许的操作符，恰好达到上限。这体现了与非运算构造其他布尔逻辑的思想。

P3 — negativePart

目标：当 x 为负数时返回其相反数，否则返回 0。

先对 x 作算术右移 31 位：非负数得到全 0 掩码，负数得到全 1 掩码。再通过补码恒等式 $-x = \sim x + 1$ 构造相反数，最后用掩码选择所需结果：

```
return (x >> 31) & (~x + 1);
```

当 $x = \text{INT_MIN}$ 时，补码意义下的取负仍保留 `0x80000000`，符合本实验 32 位位向量模型。

P4 — copyByteWithin

目标：复制 x 中的第 src 个字节，覆盖第 dst 个字节，其余位保持不变。

一个字节占 8 位，因此字节编号对应的起始位移量为 $\text{src} \ll 3$ 和 $\text{dst} \ll 3$ 。首先构造目标字节的反向掩码 $\sim(0xff \ll \text{shift_dst})$ ，清除旧目标字节；再将 x 算术右移到源字节所在位置，用 $\& 0xff$ 截取这 8 位，将其左移到目标字节位置。最后通过按位或合并保留部分和新字节。

实现中的关键式为：

```
old_byte = x & ~(0xff << shift_dst);
new_byte = ((x >> shift_src) & 0xff) << shift_dst;
return old_byte | new_byte;
```

要点：先右移，再使用 `0xff` 掩码，可以去掉算术右移带来的符号扩展。

P5 — `logicalShift`

目标：在 C 语言默认算术右移的模型上实现逻辑右移，即左侧统一补 0。

直接使用 `x >> n` 时，负数左侧会补 1。解决办法是另外构造一个掩码，将补入的高位全部清除：

```
mask = ~(((1 << 31) >> n) << 1);
return (x >> n) & mask;
```

`1 << 31` 先形成最高位单独为 1 的数；算术右移 `n` 位后，再左移 1 位，就会得到需要清除的高位区域。取反后便留下逻辑右移应当保留的低位。`n=0` 和 `n=31` 也被此式涵盖。

P6 — `swapNibblePairs`

目标：交换每个字节内部的高 4 位和低 4 位。

先构造重复的 `0x0F0F0F0F` 掩码：从 `0x0F` 出发，依次与左移 8 位、16 位的自身作或。通过 `x & mask` 得到每个字节的低半字节，将其左移 4 位；再将 `x` 右移 4 位并与相同掩码相与，留下原来的高半字节。合并即可完成交换。

```
return ((x & mask) << 4) | ((x >> 4) & mask);
```

以 `0xAB` 为例，得到 `0xBA`。由于最终使用了按位掩码，算术右移产生的额外符号位不会进入结果。

P7 — `secondLowestZeroBit`

目标：定位从低位起第二个 0 所在的位置，并返回只有该位为 1 的掩码。

首先令 `y = ~x`，将原来的 0 位转成 1 位。对非零整数 `y`，表达式 `y & (y - 1)` 可以清除最低的 1 位；在题目允许的运算中，`y - 1` 需写作 `y + ~0`。接下来利用 `z & (-z)` 提取剩余的最低 1 位，其中 `-z` 写作 `~z + 1`。

```
y = ~x;
z = y & (y + ~0);
return z & (~z + 1);
```

若不足两个 0，清除最低 1 后会得到 0，因此最终返回 0。

P8 — `oddParity`

目标：当 32 位中 1 的数量为偶数时返回 1，否则返回 0。

异或具有“相同为 0、不同为 1”的性质，连续异或可以计算多个 bit 的奇偶性。逐步将位序列折叠：先与右移 16 位的自身异或，再分别右移 8、4、2、1 位并异或。这样全部 32 位的奇偶性最终汇聚到最低位。

```

x ^= x >> 16;
x ^= x >> 8;
x ^= x >> 4;
x ^= x >> 2;
x ^= x >> 1;
return !(x & 1);

```

最低位为 1 代表 1 的数量为奇数，题目要求偶数返回 1，所以最后取逻辑非。

P9 — rotateRightBits

目标：将 32 位数据向右循环旋转 n 位，而不是普通右移丢弃低位。

先令 $n = n \& 31$ ，相当于对 32 取余，避免移位量超过 31。循环右移由两部分拼合：一部分是原数据逻辑右移 n 位，另一部分是原数据向左移动 $32-n$ 位以补回从右边移出的 bit。由于 $n=0$ 时不能左移 32 位，可使用 $\text{shift} = (\sim n + 1) \& 31$ ，让对应移位量变为 0。

```

n = n & 31;
mask = ~((1 << 31) >> n) << 1;
shift = (~n + 1) & 31;
return ((x >> n) & mask) | (x << shift);

```

这里沿用了 P5 的逻辑右移掩码，避免算术右移引入多余的 1。

P10 — roundEvenPow2

目标：将非负数 x 舍入为最接近的 2^n 的倍数；恰在中点时，选择商为偶数的倍数。

设 $x = q \cdot 2^n + r$ ，其中 $q = x \gg n$ ， $0 \leq r < 2^n$ 。判断舍入方向的关键在于半个区间 $\text{half} = 2^{(n-1)}$ ：小于中点应向下，大于中点应向上；正好在中点时，取决于 q 的奇偶。

代码采用“加偏置后截断”的方法：

```

half = 1 << (n + ~0);
q = x >> n;
add = half + ~0 + (q & 1);
return ((x + add) >> n) << n;

```

当 q 为偶数，偏置为 $\text{half} - 1$ ，中点会向下；当 q 为奇数，偏置为 half ，中点会向上。截断最后 n 位即可得到满足 ties-to-even 规则的倍数。

P11 — midpointTowardFirst

目标：计算 x 和 y 的数学中点，避免加法溢出；当中点是半整数时，选择更靠近第一个参数 x 的整数。

直接计算 $(x+y)/2$ 可能因 $x+y$ 溢出而出错。代码使用以下恒等式得到向负无穷取整的平均值：

```

mid = (x & y) + ((x ^ y) >> 1);

```

其中 $x \& y$ 保留共有位， $x \wedge y$ 则处理不同位，通过右移形成不会先计算 $x+y$ 的平均值。若 x 与 y 的最低位不同，说明精确中点恰为半整数，需要决定是否将 `mid` 加 1。代码计算 `x_gt_y`：当二者符号不同，用符号比较；符号相同时，通过差值的符号判断顺序，从而避免比较所用的差值溢出。最终：

```
fix = (x ^ y) & 1;
return mid + (fix & x_gt_y);
```

若中点为半整数且 $x > y$ ，就取较大的整数；其他情况下维持向下的结果。

P12 — `isBetweenEitherOrder`

目标：不论区间端点 a 、 b 谁大谁小，判断 x 是否落在含端点的闭区间内。

区间判断可以写为 $(a \leq x \ \&\& \ x \leq b) \ || \ (b \leq x \ \&\& \ x \leq a)$ 。受限于不能直接使用比较运算与逻辑连接符，当前实现先提取 x 、 a 、 b 的符号位，再构造 $x-a$ 、 $x-b$ 、 $a-x$ 、 $b-x$ 四个差值。对于不同符号的比较，直接根据符号确定大小；同符号时才依据差值的符号作判断。最后用位与和位或组合出两个方向的闭区间条件。

当前版本的 `btest` 功能检查无错误，但表达式展开较多，运算符数量为 57，超过允许的 48。说明功能正确不代表符合提交规则；后续需要复用比较中的中间结果、减少重复表达式。

P13 — `mul5Sat`

目标：计算 $5x$ ；若超过 32 位有符号整数范围，则返回 `INT_MAX` 或 `INT_MIN`。

乘法部分可以通过移位与加法实现： $(x \ll 2) + x$ 。为了判断结果是否越界，代码先构造边界值 `0x19999999`（即 `INT_MAX / 5` 的整数部分），随后计算表示正溢出与负溢出的标志，并准备饱和值 $\sim(1 \ll 31)$ 和 $1 \ll 31$ 。

这一思路意在将**正常结果**与**饱和值**按条件组合，但目前存在边界处理错误。终端中的失败用例为：

```
mul5Sat(-2147483648)
实际返回: 1 (0x00000001)
预期返回: -2147483648 (0x80000000)
```

从当前代码可见，比较边界时仍可能发生 32 位运算回绕，而且 `pos_over`、`neg_over` 是 0/1 标志，却被直接用于对整个饱和值按位与，不能形成完整的全 0/全 1 选择掩码。因此该题目目前**满足 27/30 次操作的限制，但没有通过功能测试**。改进时需要区分布尔标志和位掩码，同时避免判断溢出时自身发生溢出。

P14 — `classifyAdd3`

目标：判断三个 32 位整数的精确数学和 $x+y+z$ 与有符号整数范围的关系：大于 `INT_MAX` 返回 1，小于 `INT_MIN` 返回 -1，其余返回 0。

代码先形成低 32 位的总和，并取得三个操作数及结果的符号：

```
sum = x + y + z;
sx = x >> 31;
sy = y >> 31;
sz = z >> 31;
ss = sum >> 31;
```

随后分别为全同号和两同号等符号组合构造 `pos`、`neg`、`pos2`、`neg2` 条件，期望通过结果符号与操作数符号之间的变化判断超界方向。这是尝试把**精确和的范围分类**转化为位级判断的思路。

但三个数相加可能经历一次以上的回绕，仅根据总和符号及这些有限的组合条件不足以判断精确和。例如终端显示：

```
classifyAdd3(INT_MIN, INT_MIN, INT_MIN)
实际返回: 0
预期返回: -1
```

此外，当前代码 **使用 57 个操作符，超过 52 的上限**。因此本题同时存在功能正确性与操作次数问题。后续应采用更系统的溢出分类方式，例如分两次加法并跟踪每次带符号溢出的方向，而不只观察最终符号。

P15 — floatScaleThreeHalves

目标：仅使用整数运算，在 IEEE 754 单精度位模式上计算 $f \times 3/2$ ，采用“舍入到最近，恰在中点时取偶数”。

浮点數位模式拆为符号位 `sign`、8 位阶码 `exp` 和 23 位尾数 `frac`。代码首先保留 NaN/无穷的原有位模式；对于非规格化数，使用没有隐藏首位 1 的尾数；对于规格化数，补上隐藏的最高位 1。随后将有效数字乘以 3，并将二分之一计入二进制指数。

接着检查有效数字是否超过 24 位，通过右移归一化；记录被移除的低位 `rem`，依据“超过一半进位、恰在一半时看保留最低位是否为奇数”的规则进行舍入。如果舍入产生新的进位，还需要再次调整阶码。对于结果仍属于非规格化区间的情况，则走单独的舍入路径；对于阶码超过范围的情况，返回带原符号的无穷大。

这一实现的核心是分别处理特殊值、规格化数、非规格化数和舍入进位。当前测试截图中该题 **btest 无错误，51/60 次操作**。

P16 — floatRoundEven

目标：将浮点数舍入为最近整数，平局时舍入到偶数，再以单精度浮点位模式返回；若结果为零，应保留负零的符号。

首先分解符号、阶码、尾数，NaN 和无穷大不改动。非规格化数的绝对值均小于 1，舍入结果为带符号的 0；对于指数 $E < 0$ 的规格化数，根据其大小决定舍入到 0 或 ± 1 ：特别地，当绝对值恰好为 0.5 时按偶数规则舍入到 0。对于 $E \geq 23$ 的值，单精度的尾数已没有小数部分，可以原样返回。

当 $0 \leq E < 23$ 时，通过隐藏位构造 24 位有效数字，再根据 $\text{shift} = 23 - E$ 拆分整数部分 `I` 和舍弃部分 `F`。若 `F` 大于半单位，或者等于半单位且 `I` 为奇数，便令 `I` 加 1。最后重新编码阶码和尾数；若进位使整数部分成为更高的 2 的幂，还需要增加阶码。

该题的重点是**平局取偶、阶码变化和 ± 0 的保持**。当前测试截图中该题 **btest 无错误，39/65 次操作**。

P17 — float_i2f

目标：把有符号整数 `x` 转换为 IEEE 754 单精度浮点数的位模式，效果对应 `(float)x`，但不能直接使用浮点类型或强制类型转换。

代码先分离符号位，计算无符号形式的绝对值；当输入为 0 时直接返回 0。之后利用 16、8、4、2、1 位的分段检查，快速定位最高有效 1 的位置 `e`，从而得到浮点数的阶码 `e+127`。

将有效数字左移，使最高有效 1 对齐到第 31 位后，提取 23 位尾数；同时保存剩余的 8 个低位作为舍入信息。由于 32 位整数最多只需舍弃 8 个有效低位，检查 `round_bits` 与 `0x80` 的大小关系即可实现最近偶数舍入。若尾数加一后溢出，就将尾数清零并增加阶码。

本题兼顾了最高有效位定位、`INT_MIN` 的绝对值处理以及正确舍入。当前测试截图中该题 **btest 无错误，35/40 次操作**。

P18 — bitCount

目标：统计整数的 32 位表示中有多少个 1。

采用分组并行计数（population count）的思想。先构造五个掩码：

- `0x55555555`：合并相邻单个位，得到每 2 位的计数；
- `0x33333333`：合并相邻的 2 位组，得到每 4 位的计数；
- `0x0F0F0F0F`：合并相邻的 4 位组，得到每 8 位的计数；
- `0x00FF00FF`：合并相邻的 8 位组，得到每 16 位的计数；
- `0x0000FFFF`：合并两个 16 位组，得到全部 32 位的计数。

以 8 位数据 `ABCDEFGH` 为例，第一次通过 `01010101` 掩码分别把 `(A+B)`、`(C+D)`、`(E+F)`、`(G+H)` 相加，第二次将两个相邻组再次相加，第三次得到全部 8 位中 1 的总数。32 位只需再做两轮。每一级计数的存储位宽足够容纳该组最大可能的计数，因此不会发生组间污染。

这一方法不需要逐位循环，属于典型的 **SWAR（单寄存器内并行运算）** 思路。当前实现使用 **36/40 次操作**。

P19 — bitReverse

目标：将 32 位整数的位顺序完全颠倒。

采用分治式交换：先交换左右两个 16 位半区，再交换各 8 位区块，随后交换 4 位、2 位和单个位。以 8 位序列为例，可以表示为：`12345678 → 56781234 → 78563412 → 87654321`。

实现先建立 `0x0000FFFF`，再通过掩码与其移位版本进行异或，依次生成 `0x00FF00FF`、`0x0F0F0F0F`、`0x33333333`、`0x55555555`。每一轮都以“一个方向先移动再掩码、另一个方向先掩码再移动”的方式将分组置换并合并：

```
x = ((x >> k) & mask) | ((x & mask) << k);
```

由较大块到较小块逐级交换后，所有位都到达反向位置。该实现的掩码可重复利用，最终 **34/34 次操作**，刚好满足限制。

四、测试过程与结果

4.1 功能测试：./btest

在项目目录中执行：

```
./btest
```


根据终端截图，**当前版本总分为 96/110**。P1~P12 与 P15~P19 的功能测试没有报告错误；P13、P14 各出现失败用例。该分数仅反映此次 `btest` 输出，不能代表所有提交规则均已满足。

```

provencep@LAPTOP-5TNJK0AC ~/../DataLab % main > ./btest
Score Rating Errors Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
ERROR: Test mul5Sat(-2147483648[0x80000000]) failed...
...Gives 1[0x1]. Should be -2147483648[0x80000000]
ERROR: Test classifyAdd3(-2147483648[0x80000000],-2147483648[0x80000000],-2147483648[0x80000000]) failed...
...Gives 0[0x0]. Should be -1[0xffffffff]
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 96/110

```

图 1 `./btest` 的实际终端输出，显示 96/110 和 P13、P14 的失败用例。

4.2 操作符检查： `./check_ops.py bits.c`

在项目目录中执行：

```
./check_ops.py bits.c
```

根据终端截图，**P12 使用 57/48 次操作、P14 使用 57/52 次操作，均超过限制**；其余列出的题目均未超出各自操作次数上限。操作符检查关注规则合规性，不能替代 `btest` 对函数结果的检查。

```

provencep@LAPTOP-5TNJK0AC ~/../DataLab % main > ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 10/24 operators
midpointTowardFirst: 21/32 operators
mul5Sat: 27/30 operators
floatScaleThreeHalves: 51/60 operators
floatRoundEven: 39/65 operators
float_i2f: 35/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators

Operator check failed:
isBetweenEitherOrder: 57 operators exceeds limit 48
classifyAdd3: 57 operators exceeds limit 52

```

图 2 `./check_ops.py bits.c` 的实际终端输出，显示 P12 和 P14 超出限制。

4.3 逐题汇总

题号	函数	功能测试 (btest)	操作符检查
P1	signMask	通过	1 / 2
P2	bitXor	通过	8 / 8
P3	negativePart	通过	4 / 6
P4	copyByteWithin	通过	9 / 12
P5	logicalShift	通过	6 / 20
P6	swapNibblePairs	通过	9 / 18
P7	secondLowestZeroBit	通过	7 / 8
P8	oddParity	通过	12 / 56
P9	rotateRightBits	通过	12 / 16
P10	roundEvenPow2	通过	10 / 24
P11	midpointTowardFirst	通过	21 / 32
P12	isBetweenEitherOrder	通过	57 / 48 (超限)
P13	mul5Sat	失败	27 / 30
P14	classifyAdd3	失败	57 / 52 (超限)
P15	floatScaleThreeHalves	通过	51 / 60
P16	floatRoundEven	通过	39 / 65
P17	float_i2f	通过	35 / 40
P18	bitCount	通过	36 / 40
P19	bitReverse	通过	34 / 34

说明：“通过”仅指所提交 `btest` 截图未显示该函数的错误；表格与截图来自同一次提交材料，不表示进行了额外的形式化验证。

五、问题分析与后续改进

P12：操作次数超限。 目前的比较方法功能上可行，但四组比较展开后存在大量重复逻辑。可改进的方向是共享符号关系和比较结果，并重新统计操作符数量，在保留正确性的情况下减少至少 9 个运算符。

P13：负边界溢出。 原实现先确定乘以 5 的整数范围，再执行饱和选择，这是合理的分解方向。但判断过程本身可能发生回绕，同时将 0/1 逻辑值直接充当整字掩码，造成 `INT_MIN` 用例返回 1。应重新设计无溢出的边界判断，随后用全 0 或全 1 的掩码选择饱和值。

P14：三个数相加的溢出分类不完整。 仅观察总和的符号，无法确定精确数学和是否超范围。应分析 `x+y` 及再加 `z` 两个阶段的溢出方向，或者使用不会丢失方向信息的等价位级表示，同时压缩运算符数量至不超过 52。

这些问题体现出：**边界值正确、正常样例正确和操作符数量合法是彼此独立的检查维度**。后续应在修改代码后重新执行两条测试命令并更新截图，不能沿用旧版证据。

六、实验总结与心得

本实验最有价值的收获是认识到，位运算不仅能操作单个位，还可以构成完整的控制与算术逻辑。例如，`x >> 31` 能产生符号掩码，`~x + 1` 对应补码取负，德摩根定律可以消去受限的逻辑运算符；而 `x & (x-1)` 与 `x & (~x)` 则分别用于清除和提取最低有效的 1。

在完成 P8、P18、P19 时，我进一步理解了按位分组和逐级折叠的方法：不必逐位处理 32 个 bit，可以通过异或折叠、分组求和、层级交换，在有限操作符下完成奇偶校验、位计数和位逆序。这种“把整体问题拆成结构相同的小问题”的方法，比直接枚举每种输入更具有普遍性。

浮点题让我重新理解了浮点数不等于简单的二进制整数。符号位、偏置阶码、隐含的有效数字、非规格化表示、舍入到最近偶数以及舍入后的进位，都需要独立处理。尤其是临界点附近，必须关注舍弃的低位与最终结果的奇偶性。

本次提交的测试结果为 96/110，同时仍有功能错误与操作符超限。这提醒我：程序不仅要能通过常规测试，也要认真验证 `INT_MIN`、`INT_MAX`、0、移位边界以及舍入中点等特殊情况。今后我会先证明边界条件，再实现位级表达式，最后结合功能测试和规则检查迭代改进。

七、参考与资料说明

1. 本实验随附的 `bits.c` 中的函数定义、题目要求和 Integer/Floating Point Coding Rules。
2. 本实验提供的 `btest` 功能测试工具与 `check_ops.py` 操作符检查工具及其终端输出。
3. 本人编写的逐题实现思路记录（`思路.txt`）及最终实现文件（`bits.c`）。

本报告未另行引入外部文献的具体论述。P11~P17 的文字分析系依据所提交源代码补充整理，并非来自原思路文件的逐字记录。